

The Alberta Buck - Printable Receipt (v2)

A Plain-Text, Independently Re-Verifiable Record of an Identity-Bound BUCK Payment

Perry Kundert

2026-05-29



The cryptography of the Alberta Buck identity layer names an *identity point* $M \in \mathbb{G}_1$ and binds a transaction; the artifact a human actually holds is a *receipt*. This document specifies that receipt: a plain-text slip, sized for an 80mm thermal printer, that prints the plain-text Identity of both parties and the transaction details, and embeds – as a wrapped text payload – everything required to re-verify the payment by re-scanning the slip and re-running the wallet’s verifiers. The receipt closes two gaps the on-chain proofs leave open: it carries each party’s **canonical_identity_data** so a point M resolves to a human name (verified by recomputing $M = \text{keccak}(\text{data}) \cdot G$), and it states which on-chain facts (registry records, the **Transfer** / **SpentCoupled*** event) anchor the proof. Verification is two-tier: *offline*, re-running the proof algebra and the point→human bridge from the embedded payload alone; and *chain-anchored*, confirming the embedded records against a node. Five receipt kinds are specified – an EOA transfer from a public or a private counterparty, and the B1, A1, and A2 Note flavors. The Note kinds are producible by *either* party – the depositor’s copy or the issuer’s "I paid X" copy – because the Identity-M note payload (the **idHash** preimage material: the addressed ciphertexts and issuer-signature words) is held by both, and the receipt’s disclosed identity preimages let any verifier derive the identity scalars and re-check it deterministically. The remaining proofs are the primitives already shipped (the approve handshake + **verifiable_decrypt** for EOA, the batch Schnorr for public issuers, and for the private-issuer A2 case the recipient-blinded **verifyIssuerReenc** binding the mint verified on chain). (Decryptability, Identity)

Contents

1	What a Receipt Is (and Is Not)	3
2	Two Gaps the Proofs Leave to the Receipt	3
2.1	The point -> human bridge	3
2.2	The trust anchor	3
3	The Two-Tier Verification Model	3
4	The AB-RCPT/2 Envelope	4
5	Per-Scenario Payloads	5
5.1	coa-pub – EOA transfer from a public counterparty	5
5.2	coa-priv – EOA transfer from a private counterparty	5
5.3	note-b1 / note-a1 – Note from a public issuer	6
5.4	note-a2 – Note from a private issuer	6
5.5	Why there is no "B2" (private-issuer bearer) receipt	7
6	The Physical Artifact (ESC/POS, 80mm, 203dpi, 576px)	7
7	Verification Procedure	7
8	Wallet UX and Storage	8
9	The Whole Flow, Executed: Live EVM, Real Proofs, Both Receipts	9
9.1	Boot: a chain whose id the proofs demand	9
9.2	The A2 world: a private issuer pays an addressed recipient	10
9.3	Deploy the real stack; bind the Identities (Merkle updates on the EVM)	10
9.4	Fund, and the identity-bound approve (the real CP handshake)	10
9.5	Mint (the issuer's act)	11
9.6	Transmission verification (the recipient's act, offline)	11
9.7	Deposit (the recipient's act, on chain)	12
9.8	The two receipts, side by side	12
9.9	Tier-2: anchoring both slips to the live chain	13
9.10	All three flavors: the cost of each step	13
9.11	Shutdown	14
10	Security Considerations	14
11	Phasing	15
12	Open Questions	16

1 What a Receipt Is (and Is Not)

A BUCK receipt is a **record of a completed payment**, generated by a party to that payment and printable on a commodity thermal printer. For an EOA transfer the generator is the *recipient*; for a Note it is **either** party – the depositor prints their copy, the issuer prints the payer-side "I paid X" copy – since the Identity-M note payload travels to both at mint/spend. The receipt's **role** field records which side produced it, and the renderer marks that party "- you". It is the bilateral-disclosure half of the system's privacy promise made tangible: bulk chain-analytics cannot reconstruct who paid whom, but either party to a payment can produce a slip that names the counterparty in the clear and proves it.

- **It IS:** human-readable; names both parties' plain-text Identity (a display subset on paper, the full KYC datum in the payload); states amount, kind, time, and on-chain references; self-verifying from its own text payload; and extensible with optional informational line items.
- **It is NOT:** a bearer instrument (it redeems nothing – the value already moved, and any Note nullifier it references is already spent); a chain explorer; or a re-derivation of the KYC binding of M to a legal person (that rests on the issuer's Pointcheval-Sanders credential, assumed here). The optional line items are **not** covered by verification.

2 Two Gaps the Proofs Leave to the Receipt

2.1 The point -> human bridge

Every shipped proof names an identity *point* M . A receipt must name a *person*. The link is the preimage:

Definition 1. A party's Identity datum is the canonical JSON *canonical_identity_data* (the KYC fields, sorted-key, no-whitespace – Identity Example). Its identity scalar is $m = \text{keccak}(\text{canonical}) \bmod \text{ORDER}$ and its identity point is $M = m \cdot G$.

The receipt therefore carries each party's *canonical_identity_data*, and a verifier recomputes $\text{keccak}(\text{data}) \cdot G$ and checks it equals the named M . That single check turns a curve point into "Alice Johnson, Alberta Identity Card, Alberta, Canada."

Requirement (data availability). The generating wallet must *hold* the counterparty's *canonical_identity_data*. For a **public** identity it is published (off-chain attestation). For a **private** identity it is disclosed bilaterally when the parties approve one another (or when a Note is delivered) – the same exchange that lets each verify $M = \text{keccak}(\cdot) \cdot G$ against the credential they can decrypt. A wallet that never received the preimage can still print the *point* and the issuer reference, but not the human name (see *Open Questions*).

2.2 The trust anchor

Offline math proves the receipt is *internally* sound; it cannot prove the embedded keys and events are the *real* on-chain ones. That is a chain read. Hence two tiers, both specified below.

3 The Two-Tier Verification Model

Tier	Needs	Confirms
1 – offline	the slip's text payload only	the proof algebra verifies; both M recompute from their <i>canonical_identity_data</i> ; for Notes the
2 – anchored	tier 1 + an RPC node	the embedded <code>pk / E_addr / isPublicIdentity</code> equal the registry's for each named address; the T

Tier 1 says *"these proofs are valid and name these humans"*; tier 2 says *"and those identities and that payment are the genuine on-chain ones."* Together they are the non-deniable receipt of *Decryptability*. Per the design decision, the payload is **offline-complete**: every value tier 1 needs is embedded, and tier 2's references ride alongside so anchoring is optional, not required, to read a receipt.

Property 2. (Receipt soundness.) *If tier 1 passes, the slip's proofs are valid for the embedded public inputs and each named M is the hash of the printed Identity datum. If tier 2 also passes, those inputs are the on-chain registry records and the payment is a mined event. No party can forge a passing slip naming a counterparty who did not transact (proof unforgeability + event immutability); a generator may decline to print a receipt, but cannot print a false one.*

4 The AB-RCPT/2 Envelope

The verified **core** is a deterministic map, serialized canonically (CBOR), and printed as AB-RCPT/2. followed by base64url of the bytes, wrapped to the print width. (Text-only by decision – no QR; the payload is literally plain-text, re-typable and OCR-able, and survives a scanner-less verifier.) The receipt id is `base32(sha256(core))` truncated to a short handle. The optional line items live *outside* the core (they are unverified) and are printed in their own section, never in the payload.

Shown as JSON for readability (the wire form is CBOR):

```
{
  "v": 1,
  "type": "eoa-pub" | "eoa-priv" | "note-b1" | "note-a1" | "note-a2",
  "role": "recipient" | "issuer",    // who generated it ("issuer": Note kinds only)
  "chainid": 1,
  "contracts": { "registry": "0x..", "buck": "0x..", "notes": "0x.." },
  "payer": Party,                    // funds FROM (for a Note: the issuer)
  "payee": Party,                    // funds TO (the recipient / depositor)
  "txn": Txn,
  "note": NotePayload,               // Note kinds: the Identity-M idHash preimage
  "proof": Proof,                    // type-specific (below)
  // role-dependent self-naming / counterparty-naming vd records:
  "payee_vd": Vd,                     // role=recipient: payee's own E_addr -> payee.M
  "payer_vd": Vd,                     // role=issuer, private payer (A2): own E_addr -> payer.M
  "vd_payee": Vd                      // role=issuer, B1: eDepForIss -> payee.M
}

Party {
  "addr": "0x..",
  "kind": "public" | "private",
  "identity": "<canonical_identity_data>", // preimage; keccak->m, M=m*G
  "M": ["0x..", "0x.."],                 // named identity point
  "pk": ["0x..", "0x.."],                 // registered_pk (tier-2 anchor)
  "E_addr": { "R":["..", ".."], "C":["..", ".."] } // registered ct (private; tier-2)
}

Txn {
  "kind": "eoa-transfer" | "note-spend",
  "value": 250000000,                     // BUCK base units (6 dp)
  "timestamp": 1779999011,                 // block time
  "event": "Transfer" | "SpentCoupledB1" | "SpentCoupledA1" | "SpentCoupledA2",
  "txhash": "0x..", "block": 1234567, "logindex": 3,
  // note-spend adds:
  "mint_txhash": "0x..", "mint_block": 1234500, "nullifier": "0x.."
}

NotePayload {                          // per flavor -- the idHash preimage material
  // B1: "eDepForIss"                     (the SpentCoupledB1 event ct;
  //                                     idHash is id_hash_b1(m_iss))
}
```

```

// A1:  "eNote", "eRec"                (id_hash_a1(eNote, m_iss))
// A2:  "eNote", "eIss", "T"          (id_hash_a2(eNote, eIss, T))
}

```

proof is the witness the matching verifier consumes (the approve handshake for eoa-priv; the note opening + minted **cms[]** + the public issuer's batch Schnorr + nullifier/face for the Note kinds; an A2 receipt additionally carries the mint's **issuer_reenc** binding). The vd records carry the generating side's self-naming – a **verifiable_decrypt** of one's *own* registered **E_addr** revealing one's **M** – and, for the issuer-side B1 receipt, the issuer's verifiable decryption of the event's **eDepForIss** naming the depositor. The ISSUER's leg needs no party proof – its preimage is disclosed, its scalar derivable, and B1/A1 bind it into the leaf.

The ADDRESSED legs do, and the asymmetry is the security property rather than an inconvenience. Those ciphertexts are keyed to the recipient's mailbox key, so no verifier derives the opening secret from a disclosed record – and neither does a harvester, which is the whole reason the key moved off the Identity. Each side therefore proves what only it can: the recipient by verifiable decryption under **pkRecv**, the issuer by disclosing the mint randomness for any verifier to re-encrypt with. A receipt whose legs ANY reader could reproduce is not evidence that a payment happened.

What ties that key to a person is the **binding**: a mailbox leaf, over the two POINTS, checked with a Poseidon and a path and no secret at all. An issuer-side addressed receipt without it names a mailbox rather than a person, and prints **UNVERIFIED RECIPIENT**.

5 Per-Scenario Payloads

For the EOA kinds the wallet owner is the **payee**; a Note receipt is generated from **either** side (the **role** field). Each row names the tier-1 re-check. All Note flavors use the unified spend nullifier $\text{Poseidon}(T_{NF}, \rho, \text{idHash})$ (the shipped **spend.circom** derivation) and anchor at the **Minted** and **SpentCoupled*** events.

type	payer	party-naming (tier-1 re-check)
eo-pub	EOA, public	none – payer.M recomputed from public identity ; pk anchors addr
eo-priv	EOA, private	approve handshake cp_proof + payee's vd of E_for_spender
note-b1	issuer, public	idHash = id_hash_b1(m_iss) + batch Schnorr; issuer side: vd of eDepForIss names depositor
note-a1	issuer, public	idHash = id_hash_a1(eNote, m_iss) + Schnorr; recipient: vdNote = vdRec under pkRecv ; issuer: rNote = rId re-e
note-a2	issuer, private	idHash = id_hash_a2(eNote, eIss, T) ; the same per-role legs (vdIss / rId); + issuer_reenc binding and C_{iss} –

5.1 eoa-pub – EOA transfer from a public counterparty

Lightest: the payer's **identity** is public, so **payer.M** is recomputed from it and no payer proof is needed; the payer **pk** ties the **Transfer.from** address to that public record (tier 2). Payload = payer Party (public) + payee Party + payee **vd_proof** + the **Transfer** anchor.

5.2 eoa-priv – EOA transfer from a private counterparty

The **ApproveReceipt** half: at approve time the payer re-encrypted their **M** for the payee (**E_for_spender**) and proved it (**cp_proof** = **verifyApprove**); the repaired transcript binds the registry, chain, sender, and spender. The receipt retains that historical registry address, rather than consulting a later system-wide default. The payee verifiably decrypts the ciphertext to **payer.M** (**vd_proof**). Payload = both Parties (private) + **ApproveReceipt** proofs + payee self **vd_proof** + **Transfer** anchor + registry **pk** / **E_addr** for both (tier 2).

5.3 note-b1 / note-a1 – Note from a public issuer

The four-link chain, now Identity-M-bound: the opening recomputes `cm`, `cm` is in the mint `cms[]`, the issuer is bound *into the leaf itself* (the verifier derives m_{iss} from the disclosed issuer preimage and requires `opening.idHash` to recompute as `id_hash_b1(m_iss) / id_hash_a1(eNote, m_iss)`) as well as over the batch (the Schnorr, whose Fiat-Shamir transcript folds in the issuer *account* + *chainid* – so the printed `acct` is provably the mint’s `msg.sender`, the account whose BUCK funded the note), and the nullifier was burned in the `SpentCoupledB1 / SpentCoupledA1` event paying the payee.

The payee leg differs by flavor. **A1** is addressed: `eNote` encrypts the face and `eRec` the recipient’s Identity, both under the recipient’s receiving key `pkRecv`, which the receipt states. The verifier derives nothing that opens them; each side proves what only it can. A recipient-side receipt carries verifiable decryptions under `pkRecv` (`vdNote`, `vdRec`) of vG and M_{rec} ; an issuer-side receipt discloses the mint randomness (`rNote`, `rId`), and the verifier re-encrypts and compares, since the ciphertexts are fixed by the `idHash` the spend consumed. The *mailbox binding*, when carried, ties `pkRecv` to the named M_{rec} with a Poseidon and a path; an issuer-side receipt without it names the mailbox it paid and prints **UNVERIFIED RECIPIENT**. **B1** is bearer: the depositor is named at spend – the `SpentCoupledB1` event publishes `eDepForIss`, the depositor’s Identity re-encrypted under the public issuer’s registered key (proven well-formed on chain by `verifyDepositorBinding`) – so the *issuer-side* receipt names the depositor by verifiably decrypting it (`vd_payee`), while the *recipient-side* receipt self-names via `payee_vd` as usual.

5.4 note-a2 – Note from a private issuer

The hard case: both parties are encrypted-Identity. The issuer encrypted the face (`eNote`) and its *own* registered Identity (`eIss`) under the recipient’s receiving key, and `opening.idHash = id_hash_a2(eNote, eIss, T)` commits both with the mint binding’s blinded T . The addressed legs are proven per role exactly as for A1: the recipient by verifiable decryption under `pkRecv` (`vdNote`, `vdIss`), the issuer by disclosing the mint randomness. Anti-framing – that `eIss` carries the issuer’s *registered* credential, so the recovered Identity is the true minter, never a victim – is the **issuer_reenc binding** the issuer shipped at mint, verified on chain by the A2 mint overload (`IdentityRegistry.verifyIssuerReenc`, with the mint proof exposing each leaf’s `eIss` and T) and re-verified offline here; its transcript folds in the issuer *account* + *chainid*, anchoring the printed `acct` to the mint’s `msg.sender`.

The binding alone binds `eIss` to a key the minter chose, and an ElGamal ciphertext does not commit to its key: a minter could key `eIss` so that its hidden key opens it to its own Identity and the recipient’s opens it to a registered sock puppet. So the receipt also checks the key tie, $C_{iss} - T + \gamma H_{\text{PEDERSEN}} = \text{payer.M}$, with the γ the delivery carried: the binding’s plaintext is the Identity the receipt names. The spend enforced the same tie on chain, in the A2 fold.

Payload = both Parties (private) + the **note** payload (`eNote`, `eIss`, T , γ) + the **issuer_reenc** binding + the per-role legs + the generator’s self-naming (`payee_vd` or `payer_vd`) + mint and spend anchors. The chain saw only the *blinded* binding (T hides M_{iss} without γ , Q hides the recipient’s key); no un-blinding scalar is disclosed on chain. A receipt assembled *without* the binding still prints, with `M_iss` named but stamped **UNVERIFIED ISSUER** – a narrow case, since every chain-minted A2 leaf carries a verified binding (the mint reverts otherwise).

5.5 Why there is no "B2" (private-issuer bearer) receipt

A2 is the *only* private-issuer receipt kind because A2 is the only private-issuer note kind. A private-issuer *bearer* note ("B2") is impossible, not merely unimplemented: **eIss** is the issuer's Identity encrypted under **the recipient's** receiving key, openable only with its secret k – and a bearer note has no recipient at mint for the issuer to address. Any binding the issuer could instead attach to "whoever bears the note" would be openable by every holder of the freely-circulating note (no privacy over a public B1 issuer) or by no one (no receipt). So a private issuer must address the note (A2) and a bearer note must have a public issuer (B1); the mint SNARK now *binds* this (**issuerMode**, *b14e0f0*, *68e5b55*). See *Why B2 is impossible* in the Decryptability companion.

6 The Physical Artifact (ESC/POS, 80mm, 203dpi, 576px)

576 dots wide; Font A (12x24) gives **48 columns**, Font B (9x17) gives 64. The body prints at 48 cols; the base64url payload prints at Font B / 64 cols to cut its line count. Output is 7-bit ASCII only (names transliterated if needed – see *Open Questions*). ESC/POS control: ESC @ init, ESC a n align, ESC E n emphasis, GS V cut; no graphics.

The human-readable parties and amount are *derived from the verified core* – a single source of truth – so what a person reads cannot drift from what a verifier checks. Only the NOTES section is free text.

[illegible]

7 Verification Procedure

albertabuck verify [-|FILE] [--rpc URL]:

1. **Parse.** Strip to AB-RCPT/2.END, base64url-decode, decode the canonical bytes to the core map; reject unknown `v / type / role`.
2. **Tier 1 (always).** For payer and payee: recompute $M' = \text{keccak}(\text{identity}) \cdot G$ and require $M' = M$. Check the generating side's self-naming per `role` (recipient: `payee_vd`; issuer: `payer_vd` for a private payer). Dispatch on `type`: `eoapub` needs no payer proof; `eoapriv` re-runs the approve handshake + the payee's `vd` of `E_for_spender`. For the Note kinds derive m_{iss} from the disclosed issuer preimage – never the payee's, which would open its mail – and require: the opening's $\text{cm} \in \text{cms}[]$; `opening.idHash` recomputes from the `note` payload (`id_hash_b1/a1/a2`); the addressed legs hold under `pkRecv` per `role` (recipient: `vdNote` and `vdRec / vdIss`; issuer: the disclosed mint randomness re-encrypts to the committed ciphertexts); the mailbox binding, when carried, folds to its stated root; the public issuer's batch Schnorr (B1/A1) or the `issuer_reenc` binding (A2) verifies against the embedded registry record and the printed issuer `acct`, and for A2 $C_{iss} - T + \gamma H_{\text{PEDERSEN}} = \text{payer.M}$; the nullifier equals $\text{Poseidon}(T_{NF}, \rho, \text{idHash})$; and `face = opening.v = txn.value`. For an issuer-side B1 receipt also verify `vd_payee` over the event's `eDepForIss`. Require the named `M`, `value`, and addresses to match the human-readable fields the renderer derived.
3. **Tier 2 (--rpc).** For each named address read `_pk`, `_E_addr`, `isPublicIdentity` and assert equality with the payload; read the log at (`txhash`, `logIndex`) and assert the `event` (`Transfer / SpentCoupled*`), `value / face`, `recipient / from / to`, and (for Notes) `nullifier` match, the Minted event at `mint_txhash` carries `cms[]` from the printed issuer `acct` (= the `transferFrom` source that funded the note), and (B1) the spend's `eDepForIss` equals the payload's; assert the block is mined and final.
4. **Report.** `VALID` (+ named identities, amount) / `INVALID (reason)` / `UNVERIFIED ISSUER` for an A2 slip carrying no binding.

8 Wallet UX and Storage

The receipt is generated from a selected payment:

1. **Select** a row in tx history where the owner is `to` (a `Transfer`) or `recipient` (a `SpentCoupled*`); an issuer instead selects a spend of a note it minted (for B1, scanning `SpentCoupledB1` logs for its own `issuer` address and decrypting each `eDepForIss`).
2. **Gather** per the scenario table: a registry read (public payer), stored approve material (private EOA), or the stored note artifact – the `opening`, the Identity-M payload (`eNote / eRec / eIss / T`), and the mint reference; plus each party's `canonical_identity_data` from wallet storage.
3. **Self-name:** `verifiable_decrypt` the owner's own `E_addr` (and, B1 issuer side, the event's `eDepForIss`).
4. **Anchor:** read the `Transfer / SpentCoupled*` event and registry records.
5. **Prove + assemble:** generate the `vd` record(s) with `sk_owner`, build the core, *verify it locally*, set the status banner.
6. **Render + print:** ESC/POS at the configured width; or emit plain text.

Storage the wallet must retain (beyond keys): the owner's (sk, identity_data); per approved counterparty (addr, identity_data, E_for_me, cp_proof); per received *or minted* Note the opening + the Identity-M payload and its mint reference (mint_txhash, cms or cm index, issuer addr, issuer_sig | issuer_reenc binding); and an event source (indexer or RPC) for the Transfer / SpentCoupled* logs and registry reads. Note that this is exactly the material that already travels with a note off chain – the issuer keeps what it created at mint, the recipient what it was handed – so both-party generation needs no extra exchange.

CLI surface (proposed). `albertabuck receipt <txref> [--width 48|64] [--escpos|--text]` to produce a slip; `albertabuck verify [-|FILE] [--rpc]` to check a scanned payload. Both reuse the wallet library; `verify` has no wallet-secret dependency (a third party runs it).

9 The Whole Flow, Executed: Live EVM, Real Proofs, Both Receipts

Everything this document specifies **runs**. The blocks below are org-babel source blocks sharing one Python session: re-exporting this document (`make nix-venv-doc-receipt`) re-runs them top to bottom against a freshly spawned local EVM, and the printed **RESULTS** are regenerated. The worlds come from the committed end-to-end fixtures (`alberta_buck/test/vectors/e2e/{a1,a2,b1}.json`, built by `make nix-snark-e2e-fixtures` and shipped as package data): one mutually-consistent world per Note flavor with a **real Groth16 proof at every gate** – batch mint, spend, and the deposit gate (the folded proof for A1/A2, the depositor binding and its membership proof for B1) – and **real named identities**: the canonical Alice Johnson / Bob Smith KYC data of the wallet's identity vectors. Proof generation happened when the fixtures were built (a wallet does it the same way; the measured prover wall-times ride along in each fixture, and the flow companion's *Prover Bill of Materials* prices the artifacts and proving per flavor and role); everything below – registry Merkle updates, the identity-bound approve handshake, Groth16 verification, escrow and payout – is live EVM execution, exactly the path `test/NotesE2E.t.sol` drives under forge.

9.1 Boot: a chain whose id the proofs demand

Every Fiat-Shamir transcript and SNARK public input in the fixtures binds `chainid=1`, so the session runs anvil with chain-id 1; the pinned genesis timestamp (plus a fixed 12s block cadence below) makes the whole transcript – including the receipt bytes – reproducible run to run.

```
import random
from alberta_buck.sim.anvil import Anvil
from alberta_buck.sim.notes_stack import E2EFixture, NotesStack
from alberta_buck.wallet import backend
from alberta_buck.wallet.envelope import serialize_core, deserialize_core, receipt_id
from alberta_buck.wallet.verify_receipt import verify_receipt
from alberta_buck.wallet.render import render_receipt, TextDriver

assert backend() == "kernel", "must run the production kernel: evaluate in the repo venv"
print(f"wallet crypto backend: {backend()} (buck_core.buck_identity)")

_seed = random.Random(0xAB12C97)          # ONE deterministic transcript
rng = lambda: _seed.getrandbits(256)

anvil = Anvil(chain_id=1, auto_impersonate=True, timestamp=1780000000).start()
blk = anvil.w3.eth.get_block("latest")
print(f"anvil up: chain-id {anvil.w3.eth.chain_id}, genesis timestamp {blk['timestamp']}")

wallet crypto backend: kernel (buck_core.buck_identity)
anvil up: chain-id 1, genesis timestamp 1780000000
```

9.2 The A2 world: a private issuer pays an addressed recipient

The hardest flavor first. Bob Smith (a private Corporate Identity) minted a 250-BUCK note addressed to Alice Johnson's identity **point** – not to any account she owns.

```
import json
fx = E2EFixture.load("a2")
iss, dep = json.loads(fx.issuer.identity), json.loads(fx.depositor.identity)
print(f"flavor      {fx.flavor} (addressed, PRIVATE issuer); face {fx.face/1e6:.6f} BUCK")
print(f"issuer      {iss['given_name']} {iss['family_name']} ({iss['id_type']})"
      f"  acct 0x{fx.issuer.addr:040x}")
print(f"depositor   {dep['given_name']} {dep['family_name']} ({dep['id_type']})"
      f"  acct 0x{fx.depositor.addr:040x}")
print(f"prover wall-times (offline, at mint/spend): {fx.timings}")

flavor      a2 (addressed, PRIVATE issuer); face 250.000000 BUCK
issuer      Bob Smith (Corporate Registration)  acct 0x0a11ce000000000000000000000000000000a11ce
depositor    Alice Johnson (Alberta Identity Card) acct 0xde9051700000000000000000000000000000de90517
prover wall-times (offline, at mint/spend): {'mint_prove_s': 0.956, 'spend_prove_s': 0.746, 'deposit_gate_prove_s': 14.744}
```

9.3 Deploy the real stack; bind the Identities (Merkle updates on the EVM)

The full contract stack – IdentityRegistry with the incremental Poseidon identity accumulator, the Buck stack, Notes, and the **real** generated Groth16 verifiers behind their adapters – then the two identity binds. Each bind appends the party's identity leaf; the contract's incrementally-maintained root must replay the wallet tree's.

```
stack = NotesStack(anvil, fx, rng=rng, block_time=12)
binds = stack.bind_identities()
for s in binds:
    print(f"{s.name:26s} gas={s.gas:>9,}")
onchain = stack.reg.functions.identityRoot().call()
print(f"identityRoot (EVM-incremental Poseidon) = {hex(onchain)[:20]}...")
print(f" == the wallet's IdentityTree root:    {onchain == int(fx.raw['identityRoot'])}")

authorize issuer binding    gas=    55,025
bind issuer                 gas=   521,991
authorize depositor binding gas=    55,025
bind depositor              gas=   522,039
setRootAuthority            gas=    47,610
setAggregator               gas=    48,821
postIdentityRoot            gas=   118,126
identityRoot (EVM-incremental Poseidon) = 0x6d343060b877f120f2...
== the wallet's IdentityTree root:    True
```

9.4 Fund, and the identity-bound approve (the real CP handshake)

The issuer is granted BuckCredit and mints BUCK; then each private party identity-approves the Notes pool with the **real** Chaum-Pedersen re-encryption proof (Buck.**approve** / IdentityRegistry.**verifyApprove**) – the same mutual-decryptability handshake an EOA transfer uses, here providing the escrow allowance and the pool's receipt fragments. No storage hacks: every step is the legitimate path.

```
for s in stack.fund_issuer():
    print(f"{s.name:26s} gas={s.gas:>9,}")
for party, amount, name in ((fx.issuer, 2 * fx.face, "approve issuer->pool"),
                             (fx.depositor, 0, "approve depositor->pool")):
    s = stack.approve_pool(party, amount, name)
    print(f"{s.name:26s} gas={s.gas:>9,}    (CP proof: verifyApprove)")
```

```

acceptIssuer          gas=   46,654
createCredit          gas=  198,342
activateCredit        gas=   34,402
buck.mint              gas=  101,859
approve issuer->pool    gas=  196,471  (CP proof: verifyApprove)
approve depositor->pool gas=  155,169  (CP proof: verifyApprove)

```

9.5 Mint (the issuer's act)

The issuer submits the real `mint_batch_a2` Groth16 proof with the per-leaf re-encryption binding; the contract verifies both, escrows the face, and installs the SNARK-attested new note root.

```

mint = stack.mint()
print(f"Notes.mint tx {mint.txhash[:20]}.. block {mint.block} "
      f"gas {mint.gas:,} ({mint.seconds:.3f}s on anvil)")
print(f" proof was generated offline by the issuer in "
      f"{fx.timings['mint_prove_s']}s (Groth16, mint_batch_a2 N=1)")
pool = stack.buck.functions.balanceOf(stack.notes.address).call()
print(f" escrowed: pool balance = {pool/1e6:.6f} BUCK")

Notes.mint tx 0xf768232909f1bb1f3c.. block 40 gas 671,961 (0.109s on anvil)
proof was generated offline by the issuer in 0.956s (Groth16, mint_batch_a2 N=1)
escrowed: pool balance = 250.000000 BUCK

```

9.6 Transmission verification (the recipient's act, offline)

Alice receives the note off chain – the opening, the payload (`eNote`, `eIss`, `T`) with γ wrapped to her mailbox, the binding, and the mint reference – and verifies it with **one secret she already owns**: her receiving secret k . Her identity scalar opens nothing. A receipt verifier later re-runs these checks from the proofs she attaches, never from her secret.

```

from alberta_buck.wallet.bn254 import G1, add, mul, neg, eq
from alberta_buck.wallet.num import H_PEDERSEN
from alberta_buck.wallet.elgamal import elgamal_decrypt
from alberta_buck.wallet.notes import id_hash_a2, note_commitment
from alberta_buck.wallet.issuer_reenc import issuer_reenc_verify
from alberta_buck.wallet.recvkey import verify_receiving_binding

nc = fx.note_cts()
k_recv = int(fx.raw["parties"]["depositor"]["kRecv"])
m_rec = fx.depositor.m
gamma = int(fx.raw["issuerSecrets"]["gamma"])
print(f"idHash == id_hash_a2(eNote, eIss, T): "
      f"{id_hash_a2(nc['eNote'], nc['eIss'], fx.binding.T) == fx.opening.id_hash}")
print(f"Dec(eNote, k_recv) == v * G: "
      f"{eq(elgamal_decrypt(nc['eNote'], k_recv), mul(G1, fx.opening.v))}")
print(f"Dec(eIss, k_recv) == issuer's M: "
      f"{eq(elgamal_decrypt(nc['eIss'], k_recv), fx.issuer.M)}")
print(f"Dec(eIss, m_rec) == issuer's M: "
      f"{eq(elgamal_decrypt(nc['eIss'], m_rec), fx.issuer.M)} (the Identity is NOT a key)")
print(f"mailbox binding ties pk_recv to M: "
      f"{verify_receiving_binding(fx.depositor.M, fx.mailbox_binding(), int(fx.raw['identityRoot']))}")
print(f"mint binding (anti-framing) verifies: "
      f"{issuer_reenc_verify(fx.issuer.pk, fx.issuer.E, nc['eIss'], fx.binding, fx.issuer.addr, fx.chainid)}")
print(f"C_i - T + gamma * H == issuer's M: "
      f"{eq(add(add(nc['eIss'], .C, neg(fx.binding.T)), mul(H_PEDERSEN, gamma)), fx.issuer.M)}")
print(f" (the key tie)")
print(f"cm in the Minted batch: "
      f"{note_commitment(fx.opening) in fx.cms}")

```

```

idHash == id_hash_a2(eNote, eIss, T): True
Dec(eNote, k_recv) == v*G: True
Dec(eIss, k_recv) == issuer's M: True
Dec(eIss, m_rec) == issuer's M: False (the Identity is NOT a key)
mailbox binding ties pk_recv to M: True
mint binding (anti-framing) verifies: True
C_i - T + gamma*H == issuer's M: True (the key tie)
cm in the Minted batch: True

```

9.7 Deposit (the recipient's act, on chain)

Alice spends from her registered account: the real spend Groth16 against the replayed note tree and one folded proof against a posted identity root – one atomic `spendCoupledA2`.

```

spend = stack.spend()
t = fx.timings
print(f"Notes.spendCoupledA2 tx {spend.txhash[:20]}.. block {spend.block} "
      f"gas {spend.gas:,} ({spend.seconds:.3f}s on anvil)")
print(f" proofs offline: spend {t['spend_prove_s']}s + folded gate "
      f"{t['deposit_gate_prove_s']}s")
print(f" nullifier burned: "
      f"{stack.notes.functions.nullifiers(fx.nullifier).call()}")
from web3 import Web3
bal = stack.buck.functions.balanceOf(
    Web3.to_checksum_address(f'0x{fx.payout:040x}')).call()
print(f" paid out: depositor balance = {bal/1e6:.6f} BUCK")

```

```

Notes.spendCoupledA2 tx 0x63bd53667874d71164.. block 41 gas 881,104 (0.115s on anvil)
proofs offline: spend 0.746s + folded gate 14.744s
nullifier burned: True
paid out: depositor balance = 249.999999 BUCK

```

9.8 The two receipts, side by side

Both parties now print their slips from the material each already holds – Alice her copy, Bob his payer-side "I paid X" copy – anchored to the live chain's real transaction hashes, blocks, and timestamps. The deterministic note legs are byte-identical between the two; only the self-naming proof (and the "- you" marker) differs.

```

anchors = stack.anchors(mint, spend)
dep_core = fx.build_receipt("recipient", stack.contracts, rng=rng, **anchors)
iss_core = fx.build_receipt("issuer", stack.contracts, rng=rng, **anchors)
for label, core in (("depositor (Alice)", dep_core), ("issuer (Bob)", iss_core)):
    b = serialize_core(core)
    res = verify_receipt(deserialize_core(b))
    print(f"{label:18s} receipt {receipt_id(b)} tier-1: {res.reason}"
          f" ({len(b):,} B payload)")

```

```

depositor (Alice) receipt 574ey3drxxew tier-1: VALID (11,600 B payload)
issuer (Bob)      receipt dwaeczhbx6v tier-1: VALID (9,680 B payload)

```

```

def side_by_side(a, b, gap=4):
    la, lb = a.rstrip().split("\n"), b.rstrip().split("\n")
    w, n = max(map(len, la)), max(len(la), len(lb))
    la += [""] * (n - len(la)); lb += [""] * (n - len(lb))
    return "\n".join(f"{x:<{w}}{' ' * gap}{y}" for x, y in zip(la, lb))

```

```

render = lambda c: TextDriver(48).render(render_receipt(c))
print(side_by_side(render(dep_core), render(iss_core)))

```

ALBERTA BUCK PAYMENT RECEIPT	ALBERTA BUCK PAYMENT RECEIPT
----- Receipt 574ey3drxxew -----	----- Receipt dwaeczhbx6v -----
FROM (payer) (PRIVATE IDENTITY) Bob Smith Corporate Registration - Alberta, Canada acct 0x0a11ce0000000000000000000000000000a11ce idpt 0x2dabfd96aabe..38e9d7c70dca -----	FROM (payer - you) (PRIVATE IDENTITY) Bob Smith Corporate Registration - Alberta, Canada acct 0x0a11ce0000000000000000000000000000a11ce idpt 0x2dabfd96aabe..38e9d7c70dca -----
TO (payee - you) (PRIVATE IDENTITY) Alice Johnson Alberta Identity Card - Alberta, Canada acct 0xde90517000000000000000000000000000de90517 idpt 0x2e7538b21d80..54b64a191ea4 -----	TO (payee) (PRIVATE IDENTITY) Alice Johnson Alberta Identity Card - Alberta, Canada acct 0xde90517000000000000000000000000000de90517 idpt 0x2e7538b21d80..54b64a191ea4 -----
TRANSACTION Type Note spend - A2 addressed, private issuer Amount 250.000000 BUCK When 2026-05-28 20:29 UTC Chain 1 Block 41 Log 2 Tx 0x63bd53667874..36340dc10718 Mint 0xf768232909f1..17c9888fcd90 -----	TRANSACTION Type Note spend - A2 addressed, private issuer Amount 250.000000 BUCK When 2026-05-28 20:29 UTC Chain 1 Block 41 Log 2 Tx 0x63bd53667874..36340dc10718 Mint 0xf768232909f1..17c9888fcd90 -----
VERIFICATION status @ issue: VALID receipt 574ey3drxxew ----- a record, not a bearer instrument github.com/pjkundert/alberta-buck	VERIFICATION status @ issue: VALID receipt dwaeczhbx6v ----- a record, not a bearer instrument github.com/pjkundert/alberta-buck

9.9 Tier-2: anchoring both slips to the live chain

A verifier with an RPC node confirms the embedded registry records and the event anchor are the genuine on-chain ones.

```
from web3 import Web3
dep_addr = Web3.to_checksum_address(f"0x{fx.depositor.addr:040x}")
pk_onchain = stack.reg.functions.pk0f(dep_addr).call()
print(f"registry _pk[depositor] == receipt payee.pk: "
      f"{pk_onchain[0] == int(dep_core.payee.pk['x'], 16)}")
rcpt = anvil.w3.eth.get_transaction_receipt(dep_core.txn.txhash)
print(f"spend tx mined in block {rcpt['blockNumber']} "
      f"(receipt says {dep_core.txn.block}); "
      f"log #{dep_core.txn.logindex} present: "
      f"{dep_core.txn.logindex in [l['logIndex'] for l in rcpt['logs']]}")
print(f"issuer acct in receipt == Minted.issuer: "
      f"{dep_core.payer.addr[-40:] == f'{fx.issuer.addr:040x}'}")

registry _pk[depositor] == receipt payee.pk: True
spend tx mined in block 41 (receipt says 41); log #2 present: True
issuer acct in receipt == Minted.issuer: True
```

9.10 All three flavors: the cost of each step

The same lifecycle for B1 (bearer, public issuer) and A1 (addressed, public issuer), then the per-flavor cost table: on-chain gas for the two lifecycle transactions (full-call, as a node bills them) and the offline prover wall-times each party paid. Every flavor yields a VALID receipt for **both** parties.

```
runs = {"a2": (stack, mint, spend)}
for flavor in ("b1", "a1"):
    f2 = E2EFixture.load(flavor)
    st = NotesStack(anvil, f2, rng=rng, block_time=12)
    steps = st.run_lifecycle()
    runs[flavor] = (st, steps["mint"], steps["spend"])

hdr = (f"{'flavor':8s} {'mint gas':>10s} {'spend gas':>10s} "
      f"{'mint prove':>11s} {'spend proves':>13s} {'receipts (dep / iss)':>22s}")
```

```

print(hdr); print("-" * len(hdr))
for flavor in ("b1", "a1", "a2"):
    st, m, s = runs[flavor]
    f2, t = st.fx, st.fx.timings
    prove_spend = t["spend_prove_s"] + t["deposit_gate_prove_s"]
    a = st.anchors(m, s)
    rcpts = []
    for role in ("recipient", "issuer"):
        core = f2.build_receipt(role, st.contracts, rng=rng, **a)
        rcpts.append(verify_receipt(deserialize_core(serialize_core(core))).reason)
    print(f"{flavor:8s} {m.gas:>10,} {s.gas:>10,} "
          f"{t['mint_prove_s']:>10.2f}s {prove_spend:>12.2f}s "
          f"{rcpts[0]:>11s} / {rcpts[1]}")

```

flavor	mint gas	spend gas	mint prove	spend proves	receipts (dep / iss)
b1	545,962	775,091	0.91s	20.12s	VALID / VALID
a1	545,974	886,997	1.04s	15.70s	VALID / VALID
a2	671,961	881,104	0.96s	15.49s	VALID / VALID

The mint gas difference (A2 over B1/A1) is the on-chain verification of the per-leaf re-encryption binding – the price of an *anonymous* issuer; the spend difference (A1/A2 over B1) is the note $\leftrightarrow$ eEnc= tie – the price of *addressed* ("only M_{rec} can spend"). The prover cost is dominated by the tie circuits (~2.4M-2.9M constraints); all on-chain verification is constant-gas.

9.11 Shutdown

```

anvil.stop()
print("anvil stopped. The receipts above remain verifiable offline --")
print("tier 1 needs no chain at all, only the slip's own text payload.")

```

```

anvil stopped. The receipts above remain verifiable offline --
tier 1 needs no chain at all, only the slip's own text payload.

```

10 Security Considerations

- **PII on paper.** The slip is a deliberate disclosure: anyone holding it can name both parties. The display subset (name / id_type / jurisdiction) keeps the *printed* face modest, but the payload carries each party's **full canonical_identity_data** in base64 – base64 is *encoding*, *not encryption*. A receipt must be handled like a signed KYC disclosure, not like cash.
- **No value, no replay.** The slip redeems nothing; a referenced Note nullifier is already burned, an EOA transfer already mined. Re-printing or copying a slip moves no funds.
- **Generator honesty bounds.** Tier-1/2 bind the parties, amount, and event; the free-text NOTES are unbound, so a generator may write anything there – hence they are labelled *informational*, *not verified*, and excluded from the receipt id.
- **A2 issuer privacy.** The A2 binding is *blinded*: the chain sees $T = r' pk_{recv} + \gamma H_{PEDERSEN}$ and $Q = pk_{recv} + \beta H_{PEDERSEN}$, revealing neither the issuer's M_{iss} nor the recipient's key. An unblinded T would leak $M_{iss} = C_i - T$ – and **msg.sender** at mint *is* the issuer – so the blind is load-bearing. The receipt discloses γ , deliberately: a receipt exists to name the issuer, and γ is what lets any verifier check that the Identity it names is the one the binding proved registered. The mint-SNARK **issuerMode** signal is *bound*, the per-leaf **eIss** and T are on

chain (the `mint_batch_a2` circuit exposes each committed leaf's, which `Notes.mint` matches to its binding), and the folded spend gate makes an un-nameable note un-spendable: `eIss` keyed to a throwaway key decrypts to an Identity no registry certified, and a split key fails the key tie, so the spend reverts either way. `UNVERIFIED ISSUER` therefore narrows to a receipt assembled without its binding – never a spendable chain-minted leaf.

- **Account provenance.** The printed `acct` lines are not free text. The issuer account is folded into the Fiat-Shamir transcript of its binding (the batch Schnorr for B1/A1, `issuer_reenc` for A2), so tier 1 already pins it; tier 2 confirms it is the `Minted` event's `issuer` – the `msg.sender` whose `transferFrom` funded the note. The payee account is the `SpentCoupled*` event's payout `recipient`; the recipient-side receipt ties it to the named Identity by `payee_vd` (mirroring the on-chain deposit gate – the fold, or B1's depositor binding – that authorized the spend from a registered account bound to the same identity scalar). Under Identity-M the spend account need not exist at mint – authorization keys on the *identity* – so the receipt names whichever registered account the recipient actually used, anchored by the event.
- **Trust roots.** Soundness rests on the proof algebra (tier 1), the on-chain registry + event (tier 2), and the KYC issuer's $m \leftrightarrow \text{person}$ binding (out of scope; assumed via the PS credential).

11 Phasing

1. **Phase A (shipped).** Envelope + `verify` CLI + the plain-text renderer for all five kinds. `eoA-pub`, `eoA-priv`, `note-b1`, `note-a1` are soundly bound; the receipt `core` (`envelope.py` / `build_receipt.py` / `verify_receipt.py` / `render.py`) is the shipped wallet surface.
2. **Phase B (shipped, off-chain + on-chain issuerMode + A2 leaf-tie).** `note-a2` is soundly bound: the `issuer_reenc` binding names the private issuer's *registered* Identity; the slip reads `VALID`. The mint-SNARK `issuerMode` gate is bound on chain (`b14e0f0` / `584bb52` / `68e5b55`), so a bearer leaf cannot be private at all; and the per-leaf `eIss` tie (`mint_batch_a2` + `MintVerifierA2Adapter`) pins each binding to its committed leaf, so a `PRIVATE` batch cannot float or duplicate bindings.
3. **Phase C (shipped, Identity-M).** The note kinds re-built on the identity-targeted payloads: `idHash` preimages (`id_hash_b1/a1/a2`) are conveyed in the envelope and re-checked against the disclosed identity preimages; the addressed ciphertexts (`eNote` / `eRec` / `eIss`) are keyed to the recipient's receiving key and proven per role – verifiable decryption by the recipient, disclosed mint randomness by the issuer – never decrypted under a derivable scalar; the A2 key tie is checked with the disclosed γ ; and the folded spend gate closes the same ties on chain, so a spendable note is always nameable and `UNVERIFIED ISSUER` survives only for a receipt missing its binding. Receipts are generated by **both Note parties** (`role`): recipient-side and issuer-side ("I paid X") for B1/A1/A2, the B1 issuer naming its depositor from the `SpentCoupledB1` event's `eDepForIss`. Anchors moved to the unified nullifier and the `SpentCoupled*` events.
4. **Later.** The ESC/POS byte driver (Phase A shipped the plain-text driver; a port-9100 thermal driver is a new `Driver` subclass); optional QR rendering; payer-side `EOA` receipts; on-chain `RcptVerify` if a contract ever needs to gate on a receipt.

12 Open Questions

1. **Encoding for OCR.** base64url is compact but OCR-hostile (l/1/I, O/0); RFC 4648 base32 is ~20% larger but robust to hand entry and OCR. Default base64url, or base32 for paper? (The crypto payload is high-entropy, so compression buys little.)
2. **Unicode names.** Thermal output is ASCII; non-Latin `given_name` / `family_name` need transliteration for the *printed* face, while the payload keeps the exact UTF-8 `canonical_identity_data` (the hash preimage). How to transliterate without implying the printed name is the canonical one?
3. **Point-only fallback.** When the wallet lacks a private counterparty's `identity_data` preimage, should it print the *M* fingerprint + issuer reference (de-anonymizable by the issuer under subpoena), or refuse?
4. **Payload length.** Offline-complete + text-only makes Notes / private-EOA payloads ~2-2.7 KB of base64 (~30-50 Font-B lines). Acceptable for thermal, or offer a `--refs-only` compact mode that requires `--rpc` to verify?
5. **Time and locale.** Print block timestamp (consensus truth) or wall clock, and in UTC or the wallet's locale? The core stores unix block time; the printed string is a render choice.